MACSE502 Programming for Data Science

Object-Oriented Programming in Python and Scala

Eleven OOP concepts of the syllabus (Module 1: Object-Oriented Concepts; Module 3: classes, constructors, overloading, inheritance, abstract classes, traits, companion objects and more), each shown in both languages with code that was run and its real output.

No.	Concept	Python	Scala	Seen in sample questions
1	Class, object, constructor, self and this	`class`, `__init__`, `self`	class header is the primary constructor, `def this`, `this`	M3-Q1, M1-Q3
2	Encapsulation and access control	`_x`, `__x`, `@property` (convention)	`private`, `protected`, public default (enforced)	M3-Q4
3	Inheritance and super	`class B(A)`, `super()`	`extends`, `override` required, `super`	M3-Q1
4	Method overloading and overriding, field overriding	no overloading; default args, `*args`	real overloading; `override val`	M3-Q1, M3-Q2
5	Polymorphism and operator overloading	duck typing, `__add__`	subtyping, method named `+`	M3-Q2
6	Abstract class	`ABC` + `@abstractmethod` (run-time error)	`abstract class` (compile-time error)	M3-Q2
7	Multiple inheritance, traits and mixins	multiple inheritance, MRO	traits, `with`, linearization	M3-Q5
8	final	`typing.final` (hint only)	`final` (enforced)	M3-Q2
9	Singleton, companion object and class-level members	`__new__`, `@classmethod`, `@staticmethod`	`object`, companion object, `apply`	M3-Q4
10	Case classes and data classes	`@dataclass`	`case class`	M3-Q4
11	Anonymous objects and classes	`SimpleNamespace`, `type(...)`	`new Trait { ... }`	syllabus only

Note. The last column lists where the concept appeared among the sample questions of this course (`MACSE502_Sample_Questions_Answers.pdf`). It shows what has been asked so far; it is not a prediction of the FAT paper.

Tip. How to use this file: read the one-line idea, compare the two listings, then learn the boxed "Remember" line for each concept. Python ran on version 3.10 and Scala on 2.13.12 with Java 17. The wording of some Python error messages differs in newer Python versions.

1. Class, object, constructor, self and this

A **class** is the blueprint and an **object** is one thing built from it. The **constructor** is the code that runs when the object is created and gives its fields their first values.

- **Python** has one constructor, `__init__`, and every method names the object as its first parameter, `self`. The way to get a second form is a default argument: `Student("Ravi")` starts with 0 marks.
- **Scala** writes the primary constructor in the class header: `class Student(val name: String, var marks: Int)`. `val` or `var` in front of a parameter turns it into a field. More constructors are written `def this(...)` and must start by calling another constructor.
- `this` is implicit in Scala and is written only when needed. Here `setMarks` returns `this`, so `b.setMarks(30).setMarks(55)` chains.
- A class attribute (`college`) belongs to the class and is shared: after `Student.college = "VIT Bhopal"` both objects show the new value. Scala has no `static`; shared members go into a companion object (concept 9).

Python

Python (code/oops_py.py, section 1)

```python
class Student:
    college = "VIT"                          # class attribute: one copy shared by all objects

    def __init__(self, name, marks=0):       # constructor: runs on Student(...)
        self.name = name                     # instance attributes: one copy per object
        self.marks = marks

    def grade(self):                         # self is the object the method was called on
        return "pass" if self.marks >= 40 else "fail"

    def __str__(self):                       # what print(obj) shows
        return f"{self.name} ({self.marks})"

a = Student("Asha", 72)
b = Student("Ravi")                          # default argument: the only 'second constructor' Python has
print(a, a.grade(), "|", b, b.grade())
Student.college = "VIT Bhopal"              # change the class attribute once ...
print(a.college, "|", b.college)             # ... and every object sees it
```

Output

```
Asha (72) pass | Ravi (0) fail
VIT Bhopal | VIT Bhopal
```

Scala

Scala (code/oops_scala.scala, section 1)

```scala
class Student(val name: String, var marks: Int) {        // primary constructor: the parameters after the class name
  def this(name: String) = this(name, 0)                 // auxiliary constructor: must start by calling another one
  def grade: String = if (marks >= 40) "pass" else "fail"
  def setMarks(m: Int): Student = { this.marks = m; this }   // this = the current object; returning it allows
      chaining
  override def toString: String = s"$name ($marks)"
}
val a = new Student("Asha", 72)
val b = new Student("Ravi")
println(s"$a ${a.grade} | $b ${b.grade}")
println(b.setMarks(30).setMarks(55))
```

Output

```
Asha (72) pass | Ravi (0) fail
Ravi (55)
```

Remember. Constructor = the special method that runs at object creation. Python: `__init__`. Scala: the class header and body, plus `def this` for extra ones.

2. Encapsulation and access control

Encapsulation keeps the data of an object hidden and lets other code reach it only through methods, so that rules (here: a deposit must be positive) cannot be bypassed.

- **Python** has no access keywords. One underscore (`_bank`) is only a request to other programmers. Two underscores (`__balance`) make Python store the field as `_Account__balance`: `acc.__balance` fails, but `acc._Account__balance` still returns 1500 in the output. A `@property` without a setter makes an attribute read-only.
- **Scala** has `private` (this class only), `protected` (this class and its subclasses) and public as the default, with no keyword. The compiler enforces them, so a wrong access is rejected before the program runs (compiler output below).
- The guard is `require(amount > 0, ...)` in Scala, which throws IllegalArgumentException; in Python it is an `if` and `raise ValueError`.

Python

Python (code/oops_py.py, section 2)

```python
class Account:
    def __init__(self, owner, balance):
        self.owner = owner                  # public
        self._bank = "VITB"                 # one underscore: 'protected' by convention only
        self.__balance = balance            # two underscores: stored as _Account__balance (name mangling)

    @property
    def balance(self):                      # read-only attribute
        return self.__balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("amount must be positive")
        self.__balance += amount

acc = Account("Asha", 1000)
acc.deposit(500)
print(acc.balance)
try:
    print(acc.__balance)
except AttributeError as e:
    print("AttributeError:", e)
print(acc._Account__balance)                # still reachable: Python hides the name, it does not forbid access
try:
    acc.balance = 5
except AttributeError as e:
    print("AttributeError:", e)
```

Output

```
1500
AttributeError: 'Account' object has no attribute '__balance'
1500
AttributeError: can't set attribute 'balance'
```

Scala

Scala (code/oops_scala.scala, section 2)

```scala
class Account(val owner: String, private var balance: Int) {
  protected val bank = "VITB"                          // visible in this class and its subclasses
  def deposit(amount: Int): Unit = { require(amount > 0, "amount must be positive"); balance += amount }
  def currentBalance: Int = balance                    // public getter for the private field
  private def audit(): String = s"audit of $owner"     // visible only inside Account
  def report: String = audit()                         // a public method may call a private one
}
class Savings(owner: String, start: Int) extends Account(owner, start) {
  def showBank: String = bank                          // protected: allowed here, not outside
}
val acc = new Account("Asha", 1000)
acc.deposit(500)
println(s"${acc.currentBalance} ${acc.report}")
println(new Savings("Ravi", 10).showBank)
try acc.deposit(-5) catch { case e: IllegalArgumentException => println(s"IllegalArgumentException:
    ${e.getMessage}") }
```

Output

```
1500 audit of Asha
VITB
IllegalArgumentException: requirement failed: amount must be positive
```

Code: oops_err_private.scala (a file that is meant to fail)

```
//> using scala 2.13.12
// Deliberately wrong code: this file must NOT compile. It shows what private and protected protect.
class Account(val owner: String, private var balance: Int) {
  protected val bank = "VITB"
}

object Wrong {
  def main(args: Array[String]): Unit = {
    val acc = new Account("Asha", 1000)
    println(acc.balance)        // error: balance is private
    println(acc.bank)           // error: bank is protected
  }
}
```

Compiler output: oops_err_private.scala (meant to fail): access to a private and a protected member

```
compilation FAILED (as intended)
oops_err_private.scala:10:13
  variable balance in class Account cannot be accessed as a member of Account from object Wrong
oops_err_private.scala:11:13
  value bank in class Account cannot be accessed as a member of Account from object Wrong
  Access to protected value bank not permitted because
  enclosing object Wrong is not a subclass of
  class Account where target is defined
```

Remember. Python: privacy by convention and name mangling. Scala: privacy enforced by the compiler.

3. Inheritance and super

Inheritance lets a subclass reuse a parent class, add members of its own, and replace (**override**) an inherited method.

- **Python**: `class Employee(Person)`. The child constructor calls `super().__init__(name)`. `Manager.__mro__` lists the lookup order Manager, Employee, Person, object (output).
- **Scala**: `class Employee(name: String, val salary: Int) extends Person(name)`. The arguments for the parent constructor go in the `extends` clause. Replacing a concrete method needs the word `override`; leaving it out is a compile error (below). The printed chain is Manager -> Employee -> Person -> Object.
- `Manager extends Employee extends Person` is **multilevel** inheritance in both languages. In both, `super` reaches the parent's version of a method.

Python

Python (code/oops_py.py, section 3)

```
class Person:
    def __init__(self, name):
        self.name = name

    def describe(self):
        return f"{self.name} is a person"

class Employee(Person):
    def __init__(self, name, salary):
        super().__init__(name)               # run the parent constructor first
        self.salary = salary

    def describe(self):                      # overriding: same name, new body
        return super().describe() + f" earning {self.salary}"

class Manager(Employee):                     # multilevel inheritance
    pass

m = Manager("Meena", 90000)
print(m.describe())
print(isinstance(m, Person), [c.__name__ for c in Manager.__mro__])
```

Output

```
Meena is a person earning 90000
True ['Manager', 'Employee', 'Person', 'object']
```

Scala

Scala (code/oops_scala.scala, section 3)

```scala
class Person(val name: String) {
  def describe: String = s"$name is a person"
}
class Employee(name: String, val salary: Int) extends Person(name) {   // pass name up to the parent constructor
  override def describe: String = super.describe + s" earning $salary"  // override is compulsory
}
class Manager(name: String, salary: Int) extends Employee(name, salary)
val m = new Manager("Meena", 90000)
println(m.describe)
println(m.isInstanceOf[Person])
val chain = Iterator.iterate[Class[_]](m.getClass)(_.getSuperclass).takeWhile(_ != null)
println(chain.map(_.getSimpleName.takeWhile(_ != '$')).mkString(" -> "))   // takeWhile drops the $1 that local
    classes get
```

Output

```
Meena is a person earning 90000
true
Manager -> Employee -> Person -> Object
```

Code: oops_err_override.scala (a file that is meant to fail)

```scala
//> using scala 2.13.12
// Deliberately wrong code: this file must NOT compile. Replacing a concrete method needs the word override.
class Person {
  def describe: String = "a person"
}

class Employee extends Person {
  def describe: String = "an employee"                    // error: override is missing
}

object Wrong {
  def main(args: Array[String]): Unit = println(new Employee().describe)
}
```

Compiler output: oops_err_override.scala (meant to fail): overriding without the word override

```
compilation FAILED (as intended)
oops_err_override.scala:8:7
  `override` modifier required to override concrete member:
  def describe: String (defined in class Person)
```

Remember. A Scala class has exactly one superclass. Extra behaviour comes from traits (concept 7). Python allows several parent classes.

4. Method overloading and overriding, field overriding

Overloading: the same method name with different parameters in one class, chosen at compile time.
Overriding: a subclass supplies its own body for an inherited member, chosen at run time.

- **Python has no overloading.** The second `def add` replaces the first, so `Calc().add(1, 2)` fails with the TypeError in the output. The usual substitutes are default arguments (`b=0, c=0`) and `*nums`.
- **Scala has real overloading**: the compiler picks the method by the number and types of the arguments (`add(1, 2)`, `add(1, 2, 3)`, `add(1.5, 2.5)` give 3, 6 and 4.0).
- Scala can override a field as well: `override val sound = "woof"`.
- **Trap.** The parent constructor runs first. If it uses the overridden val while it is being built (`val size = label.length`), that val is still null and the program throws NullPointerException (output). Declaring the parent's member as `lazy val` delays the calculation to its first use, and the answer is then 7, the length of "derived".

Python

Python (code/oops_py.py, section 4)

```python
class Calc:
    def add(self, a, b):
        return a + b

    def add(self, a, b, c):                  # replaces the first add: only the last def with a name survives
        return a + b + c

try:
    Calc().add(1, 2)
except TypeError as e:
    print("TypeError:", e)

class Calc2:
    def add(self, a, b=0, c=0):              # emulation 1: default arguments
        return a + b + c

    def total(self, *nums):                  # emulation 2: any number of arguments
        return sum(nums)

k = Calc2()
print(k.add(1), k.add(1, 2), k.add(1, 2, 3), k.total(1, 2, 3, 4))
```

Output

```
TypeError: Calc.add() missing 1 required positional argument: 'c'
1 3 6 10
```

Scala

Scala (code/oops_scala.scala, section 4)

```scala
class Calc {
  def add(a: Int, b: Int): Int = a + b
  def add(a: Int, b: Int, c: Int): Int = a + b + c       // same name, different number of parameters
  def add(a: Double, b: Double): Double = a + b          // same name, different parameter types
}
val k = new Calc
println(s"${k.add(1, 2)} ${k.add(1, 2, 3)} ${k.add(1.5, 2.5)}")

class Animal { val sound: String = "..."; def speak: String = s"makes $sound" }
class Dog extends Animal { override val sound: String = "woof" }      // a val can be overridden by a val
println(new Dog().speak)

// Trap: Base runs first, so while it is being built the overriding val of Derived is still null
class Base { val label: String = "base"; val size: Int = label.length }
class Derived extends Base { override val label: String = "derived" }
try new Derived() catch { case e: NullPointerException => println("NullPointerException while Derived is being
    built") }
class Base2 { val label: String = "base"; lazy val size: Int = label.length }   // fix: lazy val, computed on first
    use
class Derived2 extends Base2 { override val label: String = "derived" }
println(new Derived2().size)
```

Output

```
3 6 4.0
makes woof
NullPointerException while Derived is being built
7
```

Remember. Overloading: same class, different parameter list. Overriding: subclass, same signature. Python has only overriding.

5. Polymorphism and operator overloading

Polymorphism means one call with many behaviours: the method that runs depends on the object, not on the variable that holds it.

- In both languages the loop calls `area` on a Circle, a Rect and a base object, and each answers in its own way.
- **Python** uses **duck typing**: `Tile` is not a Shape but has `area()`, so it works. **Scala** is statically typed: the list is `List[Shape]`, so every element must be a Shape.
- **Operator overloading**: Python defines `__add__` for `+`. In Scala an operator is only a method named `+`. Both print Vec(4, 6).
- Small difference visible in the output: Rect(3, 4) prints 12 in Python (ints stay ints) and 12.0 in Scala (the parameters are Double).

Python

Python (code/oops_py.py, section 5)

```python
class Shape:
    def area(self):
        raise NotImplementedError

class Circle(Shape):
    def __init__(self, r):
        self.r = r

    def area(self):
        return round(3.14159 * self.r ** 2, 2)

class Rect(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h

    def area(self):
        return self.w * self.h

class Tile:                                  # not a Shape, but it has area(): duck typing
    def area(self):
        return 1

for s in [Circle(2), Rect(3, 4), Tile()]:    # the method that runs depends on the object, not on the variable
    print(type(s).__name__, s.area())

class Vec:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):                # operator overloading: + calls __add__
        return Vec(self.x + other.x, self.y + other.y)

    def __repr__(self):
        return f"Vec({self.x}, {self.y})"

print(Vec(1, 2) + Vec(3, 4))
```

Output

```
Circle 12.57
Rect 12
Tile 1
Vec(4, 6)
```

Scala

Scala (code/oops_scala.scala, section 5)

```scala
class Shape { def area: Double = 0.0 }
class Circle(r: Double) extends Shape { override def area: Double = math.round(3.14159 * r * r * 100) / 100.0 }
class Rect(w: Double, h: Double) extends Shape { override def area: Double = w * h }
val shapes: List[Shape] = List(new Circle(2), new Rect(3, 4), new Shape)   // one type, three kinds of object
shapes.foreach(s => println(s"${s.getClass.getSimpleName.takeWhile(_ != '$')} ${s.area}"))   // the object decides
    which area runs

class Vec(val x: Int, val y: Int) {
  def +(o: Vec): Vec = new Vec(x + o.x, y + o.y)         // an operator is only a method named +
  override def toString: String = s"Vec($x, $y)"
}
println(new Vec(1, 2) + new Vec(3, 4))
```

Output

```
Circle 12.57
Rect 12.0
Shape 0.0
Vec(4, 6)
```

Remember. Overriding plus a parent-typed variable gives run-time (dynamic) polymorphism. Overloading is compile-time polymorphism.

6. Abstract class

An **abstract class** declares members without a body that every concrete subclass must supply. It may also contain finished methods, and no object can be created from it.

- **Python**: inherit from `ABC` and mark the method `@abstractmethod`. The mistake is caught only when an object is created, at run time: both `Vehicle("x")` and `Incomplete("x")` raise TypeError (output).
- **Scala**: `abstract class`. A member without a body is abstract automatically. The compiler rejects both mistakes before the program runs. Each mistake has its own file below, because the compiler stops after the first stage that finds an error.
- `override` is optional when implementing an abstract member (`def wheels: Int = 4`). An anonymous subclass can complete the class on the spot: `new Vehicle("Hero") { def wheels: Int = 2 }`.

Python

Python (code/oops_py.py, section 6)

```python
from abc import ABC, abstractmethod

class Vehicle(ABC):
    def __init__(self, name):
        self.name = name

    @abstractmethod
    def wheels(self):                       # abstract: subclasses must provide it
        ...

    def info(self):                         # concrete method, inherited as it is
        return f"{self.name} has {self.wheels()} wheels"

class Car(Vehicle):
    def wheels(self):
        return 4

class Incomplete(Vehicle):                  # forgot to implement wheels()
    pass

print(Car("Swift").info())
for cls in (Vehicle, Incomplete):
    try:
        cls("x")
    except TypeError as e:
        print("TypeError:", e)
```

Output

```
Swift has 4 wheels
TypeError: Can't instantiate abstract class Vehicle with abstract method wheels
TypeError: Can't instantiate abstract class Incomplete with abstract method wheels
```

Scala

Scala (code/oops_scala.scala, section 6)

```scala
abstract class Vehicle(val name: String) {
  def wheels: Int                                   // abstract member: no body
  def info: String = s"$name has $wheels wheels"    // concrete member
}
class Car(name: String) extends Vehicle(name) {
  def wheels: Int = 4                               // 'override' is optional when implementing an abstract
      member
}
println(new Car("Swift").info)
val bike = new Vehicle("Hero") { def wheels: Int = 2 }   // an abstract class can be completed on the spot
println(bike.info)
```

Output

```
Swift has 4 wheels
Hero has 2 wheels
```

Code: oops_err_abstract_new.scala (a file that is meant to fail)

```scala
//> using scala 2.13.12
// Deliberately wrong code: this file must NOT compile. An abstract class cannot be instantiated.
abstract class Vehicle(val name: String) {
  def wheels: Int
}

object Wrong {
  def main(args: Array[String]): Unit = {
    val v = new Vehicle("x")                              // error
    println(v.name)
  }
}
```

Compiler output: oops_err_abstract_new.scala (meant to fail): creating an object of an abstract class

```
compilation FAILED (as intended)
oops_err_abstract_new.scala:9:17
  class Vehicle is abstract; cannot be instantiated
```

Code: oops_err_abstract_missing.scala (a file that is meant to fail)

```scala
//> using scala 2.13.12
// Deliberately wrong code: this file must NOT compile. A concrete subclass must implement every abstract member.
abstract class Vehicle(val name: String) {
  def wheels: Int
}

class Incomplete(name: String) extends Vehicle(name)       // error: wheels is not implemented

object Wrong {
  def main(args: Array[String]): Unit = println("never runs")
}
```

Compiler output: oops_err_abstract_missing.scala (meant to fail): a subclass that leaves an abstract member out

```
compilation FAILED (as intended)
oops_err_abstract_missing.scala:7:7
  class Incomplete needs to be abstract.
  Missing implementation for member of class Vehicle:
  def wheels: Int = ???
```

Remember. Abstract class = a partly finished class. Python reports misuse at run time, Scala at compile time.

7. Multiple inheritance, traits and mixins

A class can get behaviour from several sources. **Python** allows **multiple inheritance**. **Scala** allows one superclass plus any number of **traits**: interface-like units that may also contain finished methods, added by **mixin composition**.

- Scala: `class Duck(val name: String) extends Flyer with Swimmer`. A trait can also be mixed into one object at creation: `new Service with Logger`.
- Python: `class Duck(Flyer, Swimmer)`. `Duck.__mro__` shows the order in which Python looks for a method.
- **Same diamond, opposite order.** B and C both extend A. Python's `D(B, C)` prints `B>C>A`, while Scala's `D extends B with C` prints `C>B>A`. Scala's **linearization** runs the last mixin first; Python's MRO goes from left to right.
- In Scala 2.13 (used here) a trait cannot take constructor parameters; a class can. A trait can declare an abstract member (`def name: String`) that the class mixing it in fills.

Python

Python (code/oops_py.py, section 7)

```python
class Flyer:
    def fly(self):
        return f"{self.name} flies"

class Swimmer:
    def swim(self):
        return f"{self.name} swims"

class Duck(Flyer, Swimmer):                  # several parents: a class can inherit from more than one
    def __init__(self, name):
        self.name = name

d = Duck("Donald")
print(d.fly(), "|", d.swim())
print([c.__name__ for c in Duck.__mro__])    # method resolution order: left to right

class A:
    def hi(self):
        return "A"

class B(A):
    def hi(self):
        return "B>" + super().hi()

class C(A):
    def hi(self):
        return "C>" + super().hi()

class D(B, C):                               # diamond: super() follows the MRO D, B, C, A
    pass

print(D().hi())
```

Output

```
Donald flies | Donald swims
['Duck', 'Flyer', 'Swimmer', 'object']
B>C>A
```

Scala

Scala (code/oops_scala.scala, section 7)

```scala
trait Flyer { def name: String; def fly: String = s"$name flies" }
trait Swimmer { def name: String; def swim: String = s"$name swims" }
class Duck(val name: String) extends Flyer with Swimmer  // extends one, mixes in the others with 'with'
val d = new Duck("Donald")
println(s"${d.fly} | ${d.swim}")

trait Logger { def log(msg: String): Unit = println(s"log: $msg") }
class Service { def run: String = "service running" }
val s = new Service with Logger                          // mix a trait into one object when creating it
s.log(s.run)

trait A { def hi: String = "A" }
trait B extends A { override def hi: String = "B>" + super.hi }
trait C extends A { override def hi: String = "C>" + super.hi }
class D extends B with C                                 // linearization: D, C, B, A (the last mixin runs first)
println(new D().hi)
```

Output

```
Donald flies | Donald swims
log: service running
C>B>A
```

Remember. Trait = interface plus optional concrete methods. `extends` for the first parent, `with` for the others. Linearization: right to left.

8. final

final stops change: a final method cannot be overridden and a final class cannot be extended.

- **Scala** enforces it. The program in the output uses `final def pay` and `final class Leaf` normally; the two files below show the compiler refusing an override and a subclass.
- **Python** has only `typing.final`, a hint for tools such as mypy. At run time nothing is stopped: `Child().pay()` returns "changed pay" in the output.

Python

Python (code/oops_py.py, section 8)

```python
from typing import final

class Base:
    @final                                  # a hint for type checkers such as mypy; Python itself ignores it
    def pay(self):
        return "base pay"

class Child(Base):
    def pay(self):                          # allowed at run time
        return "changed pay"

print(Child().pay())
```

Output

```
changed pay
```

Scala

Scala (code/oops_scala.scala, section 8)

```scala
class Base {
  final def pay: String = "base pay"                    // cannot be overridden
  def bonus: String = "base bonus"
}
final class Leaf extends Base {                         // cannot be extended
  override def bonus: String = "leaf bonus"
}
val l = new Leaf
println(s"${l.pay} | ${l.bonus}")
```

Output

```
base pay | leaf bonus
```

Code: oops_err_final_method.scala (a file that is meant to fail)

```scala
//> using scala 2.13.12
// Deliberately wrong code: this file must NOT compile. A final method cannot be overridden.
class Base {
  final def pay: String = "base pay"
}

class Leaf extends Base {
  override def pay: String = "changed pay"                // error
}

object Wrong {
  def main(args: Array[String]): Unit = println(new Leaf().pay)
}
```

Compiler output: oops_err_final_method.scala (meant to fail): overriding a final method

```
compilation FAILED (as intended)
oops_err_final_method.scala:8:16
  cannot override final member:
  final def pay: String (defined in class Base)
```

Code: oops_err_final_class.scala (a file that is meant to fail)

```scala
//> using scala 2.13.12
// Deliberately wrong code: this file must NOT compile. A final class cannot be extended.
final class Leaf {
  def bonus: String = "leaf bonus"
}

class Twig extends Leaf                                 // error

object Wrong {
  def main(args: Array[String]): Unit = println(new Twig().bonus)
}
```

Compiler output: oops_err_final_class.scala (meant to fail): extending a final class

```
compilation FAILED (as intended)
oops_err_final_class.scala:7:20
  illegal inheritance from final class Leaf
```

Remember. final = no overriding (method) and no subclassing (class). Enforced in Scala, advisory in Python.

9. Singleton, companion object and class-level members

A **singleton** is a class with exactly one instance. In Scala a **companion object** is the object that has the same name as a class and sits in the same scope; it holds what other languages call static members.

- **Scala**: `object Registry` is a singleton by definition (no `new`; `Registry.next()` returns 1, then 2). `object Temp` is the companion of `class Temp`: its `apply` lets you write `Temp(21.5)`, and because the class constructor is `private`, the companion is the only way to make a Temp. Scala has no `static` keyword.
- **Python**: a singleton can be built with `__new__` returning the same object (`x is y` is True and both see one dictionary). Class-level members: `@classmethod` receives the class (`Temp.from_f(212)` is an alternative constructor, 100.0), `@staticmethod` receives nothing, and a class attribute (`Temp.count`) is shared.

Python

Python (code/oops_py.py, section 9)

```python
class Config:
    _instance = None

    def __new__(cls):                          # called before __init__: return the one existing object
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.values = {}
        return cls._instance

x, y = Config(), Config()
x.values["mode"] = "exam"
print(x is y, y.values)

class Temp:
    count = 0                                  # class attribute

    def __init__(self, c):
        self.c = c
        Temp.count += 1

    @classmethod
    def from_f(cls, f):                        # alternative constructor: gets the class, not an object
        return cls(round((f - 32) * 5 / 9, 1))

    @staticmethod
    def is_valid(c):                           # needs neither object nor class
        return c >= -273.15

t = Temp.from_f(212)
print(t.c, Temp.is_valid(-300), Temp.count)
```

Output

```
True {'mode': 'exam'}
100.0 False 1
```

Scala

Scala (code/oops_scala.scala, section 9)

```scala
object Registry {                                      // a singleton: exactly one instance, no 'new'
  private var count = 0
  def next(): Int = { count += 1; count }
}
class Temp private (val c: Double) {                   // private constructor: only the companion may call it
  override def toString: String = f"$c%.1f C"
}
object Temp {                                          // companion: same name, same scope as the class
  def apply(c: Double): Temp = new Temp(c)             // Temp(21.5) calls apply
  def fromF(f: Double): Temp = new Temp((f - 32) * 5 / 9)
  val absoluteZero: Double = -273.15                   // 'static'-like member
}
println(s"${Registry.next()} ${Registry.next()}")
println(s"${Temp(21.5)} ${Temp.fromF(212)} ${Temp.absoluteZero}")
```

Output

```
1 2
21.5 C 100.0 C -273.15
```

Remember. Scala: `object` = singleton. Companion = same name as the class, same file. `apply` allows `Name(args)` without `new`.

10. Case classes and data classes

A **case class** (Scala) and a **dataclass** (Python) are classes made to hold data. The language writes the constructor, the text form, equality and a copy method for you.

- **Scala**: `case class Point(x: Int, y: Int = 0)` gives `Point(1, 2)` without `new`, immutable fields, `toString` (prints Point(1,2)), `==` and `hashCode` based on the fields, `copy(y = 9)`, and pattern matching.
- **Python**: `@dataclass(frozen=True)`. Without `frozen=True` the fields can be changed; with it, assigning raises FrozenInstanceError (output). `dataclasses.replace` is the counterpart of `copy`, and `match` / `case` (Python 3.10 or newer) takes the object apart. It prints as Point(x=1, y=2).

Python

Python (code/oops_py.py, section 10)

```python
from dataclasses import dataclass, replace

@dataclass(frozen=True)                     # generates __init__, __repr__, __eq__; frozen makes it immutable
class Point:
    x: int
    y: int = 0

p = Point(1, 2)
print(p, p == Point(1, 2), replace(p, y=9))
try:
    p.x = 5
except Exception as e:
    print(type(e).__name__)
match p:                                    # structural pattern matching (Python 3.10+)
    case Point(x=1, y=yy):
        print("x is 1, y is", yy)
    case _:
        print("other")
```

Output

```
Point(x=1, y=2) True Point(x=1, y=9)
FrozenInstanceError
x is 1, y is 2
```

Scala

Scala (code/oops_scala.scala, section 10)

```scala
case class Point(x: Int, y: Int = 0)                    // no 'new', toString, equals, hashCode, copy and unapply
    for free
val p = Point(1, 2)
println(s"$p ${p == Point(1, 2)} ${p.copy(y = 9)}")
println(p.hashCode == Point(1, 2).hashCode)
p match {                                               // pattern matching takes the object apart
  case Point(1, y) => println(s"x is 1, y is $y")
  case _ => println("other")
}
```

Output

```
Point(1,2) true Point(1,9)
true
x is 1, y is 2
```

Remember. A case class is a class with `apply`, `toString`, `equals`, `hashCode`, `copy` and `unapply` (pattern matching) generated for you.

11. Anonymous objects and classes

An **anonymous class** is created without giving the class a name, for one-time use.

- **Scala**: `new Greeter { def greet(name: String) = ... }` makes an object of a nameless subclass of the trait. `new Runnable { ... }` is the common use. The output shows that the class has no name (`getSimpleName` is empty).
- **Python**: `SimpleNamespace(name="temp", size=3)` is an object with attributes and no class of your own. `type(name, bases, dict)` builds a class on the fly (here a subclass of Greeter, so `isinstance` is True). A `lambda` is an anonymous function.

Python

Python (code/oops_py.py, section 11)

```python
from types import SimpleNamespace

o = SimpleNamespace(name="temp", size=3)     # an object without a class of its own
print(o.name, o.size, o)

class Greeter:
    def greet(self, name):
        raise NotImplementedError

g = type("AnonymousGreeter", (Greeter,), {"greet": lambda self, name: f"Hello, {name}"})()   # a class made on the
    fly
print(g.greet("Asha"), isinstance(g, Greeter))
```

Output

```
temp 3 namespace(name='temp', size=3)
Hello, Asha True
```

Scala

Scala (code/oops_scala.scala, section 11)

```scala
trait Greeter { def greet(name: String): String }
val g = new Greeter { def greet(name: String): String = s"Hello, $name" }   // a class with no name, one object
println(g.greet("Asha"))
val task = new Runnable { def run(): Unit = println("running an anonymous Runnable") }
task.run()
println(s"the class has no name: ${g.getClass.getSimpleName.isEmpty}")
```

Output

```
Hello, Asha
running an anonymous Runnable
the class has no name: true
```

Remember. Anonymous class: create an object of a trait or abstract class by writing its body in braces right after `new`, with no class name.

The four pillars at a glance, and common mistakes

Pillar	Idea	Python	Scala
Encapsulation	hide the data, expose methods	`_x`, `__x`, `@property`	`private`, `protected`, getter methods
Abstraction	say what, hide how	`ABC`, `@abstractmethod`	`abstract class`, `trait`
Inheritance	reuse a parent class	`class B(A)`, `super()`	`extends`, `override`, `super`
Polymorphism	one call, many behaviours	overriding, duck typing, `__add__`	overriding, overloading, operators as methods

Watch out. Common mistakes: (1) expecting method overloading in Python, where the last `def` wins; (2) calling `__x` truly private, it is only renamed; (3) expecting `typing.final` to stop anything at run time; (4) forgetting `override` in Scala; (5) using an overridden `val` inside the parent's constructor (NullPointerException); (6) looking for `static` in Scala, which uses a companion object instead; (7) mixing up the order of `B with C` (the last mixin runs first) with Python's left-to-right MRO; (8) writing an auxiliary constructor that does not start with a call to another constructor.

Note. Everything in this file was run on this computer and the outputs are the captured outputs. The files `oops_err_*.scala` are meant to fail: they were only compiled, to capture the compiler messages. No marking scheme is known, so no marks are suggested.